

Módulo 5: Inteligência Computacional

Aplicada à Mineração de Dados

Prof. Elton Sarmanho¹
eltonss@ufpa.br

¹Faculdade de Sistemas de Informação - UFPA Campus Cametá

28 de junho de 2026

Roteiro

Lógica Fuzzy

Fundamentação

Conceitos Fundamentais

Inferência Fuzzy

Regras Fuzzy

Sistemas de Inferência Fuzzy

Agregação e Defuzzificação

Sistemas de Controle Fuzzy

Controladores Fuzzy

Roteiro

Redes Neurais

- Funções de Ativação

- Propriedades das Redes Neurais Artificiais

- Arquitetura da Rede Neural

- Perceptron

- RNA

- Multi Layer Perceptron

Deep Learning para Mineração de Dados

AutoML e Otimização de Hiperparâmetros

Estado da Arte (2020–2024)

Lógica Fuzzy

Conceitos Fundamentais

O que é Lógica Fuzzy?

- ▶ A lógica fuzzy (ou lógica difusa) é uma extensão da lógica clássica que permite graus de pertinência entre 0 e 1.
- ▶ Ela é ideal para lidar com incertezas e informações imprecisas.

Definição Formal

Conjunto fuzzy: Um conjunto A em um universo X é caracterizado por uma função de pertinência $\mu_A : X \rightarrow [0, 1]$, onde:

- ▶ $\mu_A(x) = 1$: Totalmente pertencente ao conjunto A .
- ▶ $\mu_A(x) = 0$: Não pertencente ao conjunto A .
- ▶ $0 < \mu_A(x) < 1$: Pertinência parcial.

O que é Lógica Fuzzy?

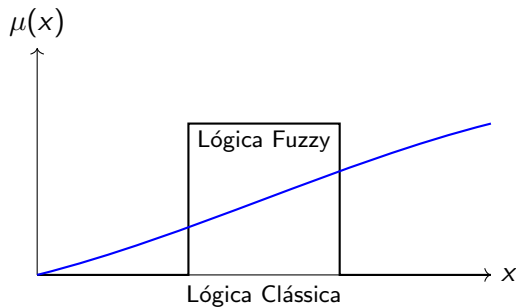
- ▶ A lógica fuzzy (ou lógica difusa) é uma extensão da lógica clássica que permite graus de pertinência entre 0 e 1.
- ▶ Ela é ideal para lidar com incertezas e informações imprecisas.

Definição Formal

Conjunto fuzzy: Um conjunto A em um universo X é caracterizado por uma função de pertinência $\mu_A : X \rightarrow [0, 1]$, onde:

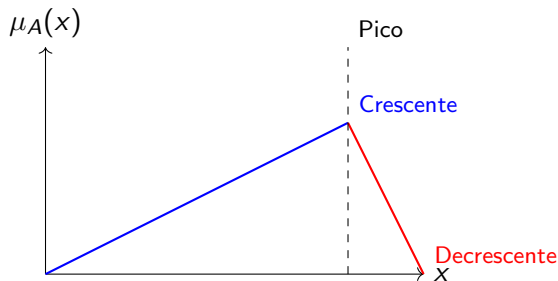
- ▶ $\mu_A(x) = 1$: Totalmente pertencente ao conjunto A .
- ▶ $\mu_A(x) = 0$: Não pertencente ao conjunto A .
- ▶ $0 < \mu_A(x) < 1$: Pertinência parcial.

Comparação: Lógica Clássica vs. Lógica Fuzzy



- ▶ **Lógica Clássica:** Valores discretos (0 ou 1).
- ▶ **Lógica Fuzzy:** Valores contínuos entre 0 e 1.

Exemplo Gráfico de Conjunto Fuzzy



- A função de pertinência $\mu_A(x)$ representa o grau de pertencimento do elemento x ao conjunto fuzzy A .

Função de Pertinência

Definição

A função de pertinência $\mu_A(x)$ de um conjunto fuzzy A mapeia cada elemento $x \in X$ em um valor no intervalo $[0, 1]$, representando o grau de pertencimento de x ao conjunto A .

▶ Exemplo de funções comuns:

- ▶ **Triangular:** $\mu_A(x) = \max \left(0, \min \left(\frac{x-a}{b-a}, \frac{c-x}{c-b} \right) \right)$
- ▶ **Trapezoidal:** $\mu_A(x) = \max \left(0, \min \left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c} \right) \right)$
- ▶ **Gaussiana:** $\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$

Função de Pertinência

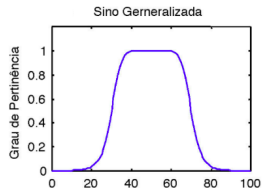
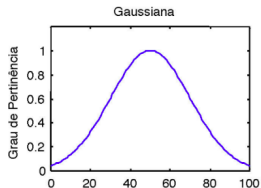
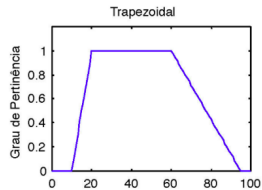
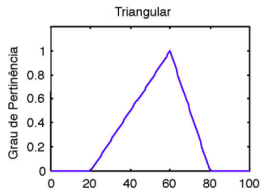
Definição

A função de pertinência $\mu_A(x)$ de um conjunto fuzzy A mapeia cada elemento $x \in X$ em um valor no intervalo $[0, 1]$, representando o grau de pertencimento de x ao conjunto A .

► Exemplo de funções comuns:

- **Triangular:** $\mu_A(x) = \max\left(0, \min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right)\right)$
- **Trapezoidal:** $\mu_A(x) = \max\left(0, \min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right)\right)$
- **Gaussiana:** $\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$

Função de Pertinência

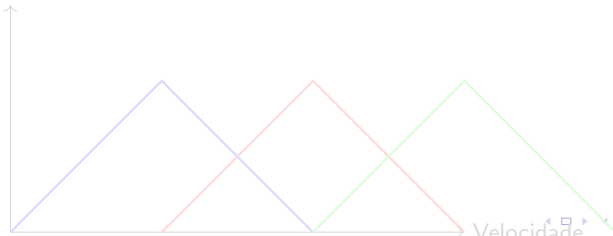


Variáveis Linguísticas

Definição: uma variável linguística é uma variável cujo valor não é um número, mas sim uma palavra ou sentença em linguagem natural.

- ▶ **Exemplo:** Velocidade de um carro
 - ▶ Valores possíveis: *lento, moderado, rápido*
- ▶ Cada valor linguístico é associado a um conjunto fuzzy.

Grau de Pertinência

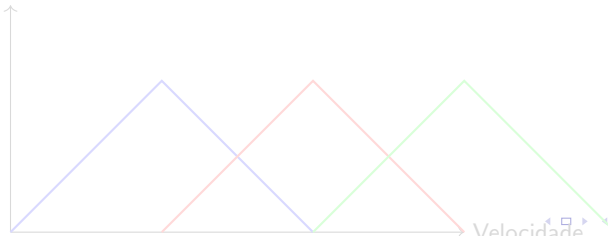


Variáveis Linguísticas

Definição: uma variável linguística é uma variável cujo valor não é um número, mas sim uma palavra ou sentença em linguagem natural.

- ▶ **Exemplo:** Velocidade de um carro
 - ▶ Valores possíveis: *lento, moderado, rápido*
- ▶ Cada valor linguístico é associado a um conjunto fuzzy.

Grau de Pertinência

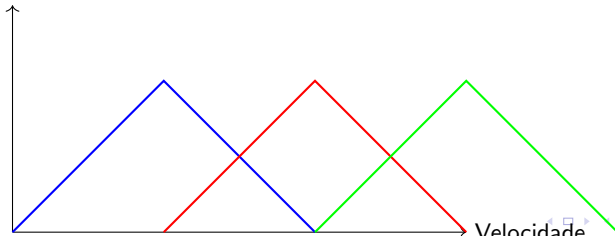


Variáveis Linguísticas

Definição: uma variável linguística é uma variável cujo valor não é um número, mas sim uma palavra ou sentença em linguagem natural.

- ▶ **Exemplo:** Velocidade de um carro
 - ▶ Valores possíveis: *lento*, *moderado*, *rápido*
- ▶ Cada valor linguístico é associado a um conjunto fuzzy.

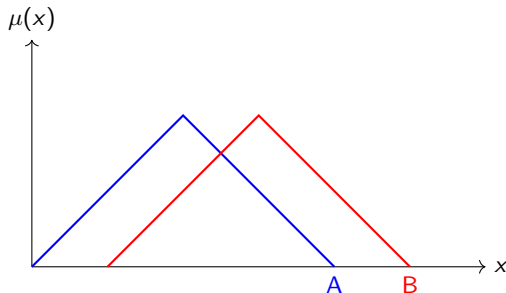
Grau de Pertinência



Operações Básicas em Conjuntos Fuzzy

Operações Fundamentais

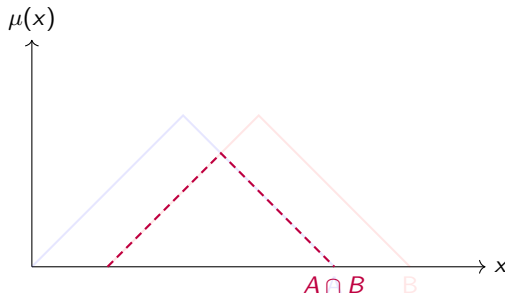
- ▶ **Interseção (AND):** $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- ▶ **União (OR):** $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- ▶ **Complemento (NOT):** $\mu_{\neg A}(x) = 1 - \mu_A(x)$



Operações Básicas em Conjuntos Fuzzy

Operações Fundamentais

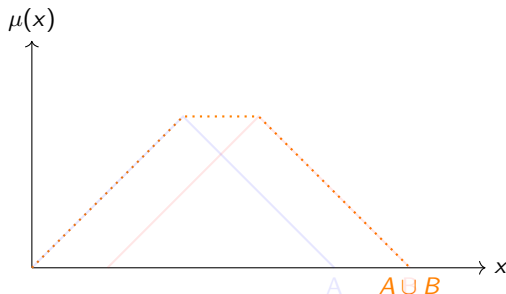
- ▶ **Interseção (AND):** $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- ▶ **União (OR):** $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- ▶ **Complemento (NOT):** $\mu_{\neg A}(x) = 1 - \mu_A(x)$



Operações Básicas em Conjuntos Fuzzy

Operações Fundamentais

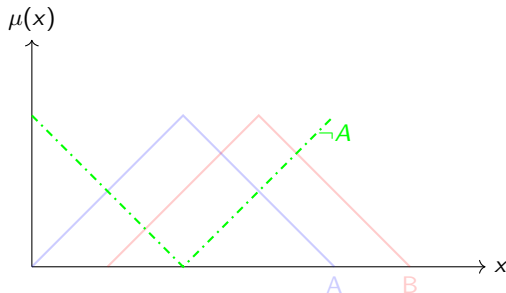
- ▶ **Interseção (AND):** $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- ▶ **União (OR):** $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- ▶ **Complemento (NOT):** $\mu_{\neg A}(x) = 1 - \mu_A(x)$



Operações Básicas em Conjuntos Fuzzy

Operações Fundamentais

- ▶ **Interseção (AND):** $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- ▶ **União (OR):** $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- ▶ **Complemento (NOT):** $\mu_{\neg A}(x) = 1 - \mu_A(x)$



Normas e Conormas (T-norms e T-conorms)

Definição

- ▶ **T-norms (Normas triangulares):** Modelam a operação de conjunção (AND) em lógica fuzzy.
- ▶ **T-conorms (Conormas triangulares):** Modelam a operação de disjunção (OR) em lógica fuzzy.

Propriedades

- ▶ **T-norm:** Monotonicidade, comutatividade, associatividade, identidade em 1.
- ▶ **T-conorm:** Monotonicidade, comutatividade, associatividade, identidade em 0.
- ▶ **Exemplo de T-norm:** $\min(\mu_A(x), \mu_B(x))$
- ▶ **Exemplo de T-conorm:** $\max(\mu_A(x), \mu_B(x))$

Normas e Conormas (T-norms e T-conorms)

Definição

- ▶ **T-norms (Normas triangulares):** Modelam a operação de conjunção (AND) em lógica fuzzy.
- ▶ **T-conorms (Conormas triangulares):** Modelam a operação de disjunção (OR) em lógica fuzzy.

Propriedades

- ▶ **T-norm:** Monotonicidade, comutatividade, associatividade, identidade em 1.
- ▶ **T-conorm:** Monotonicidade, comutatividade, associatividade, identidade em 0.
- ▶ Exemplo de T-norm: $\min(\mu_A(x), \mu_B(x))$
- ▶ Exemplo de T-conorm: $\max(\mu_A(x), \mu_B(x))$

Normas e Conormas (T-norms e T-conorms)

Definição

- ▶ **T-norms (Normas triangulares):** Modelam a operação de conjunção (AND) em lógica fuzzy.
- ▶ **T-conorms (Conormas triangulares):** Modelam a operação de disjunção (OR) em lógica fuzzy.

Propriedades

- ▶ **T-norm:** Monotonicidade, comutatividade, associatividade, identidade em 1.
- ▶ **T-conorm:** Monotonicidade, comutatividade, associatividade, identidade em 0.
- ▶ **Exemplo de T-norm:** $\min(\mu_A(x), \mu_B(x))$
- ▶ **Exemplo de T-conorm:** $\max(\mu_A(x), \mu_B(x))$

Gráfico da Interseção $A(x) \cap B(x)$

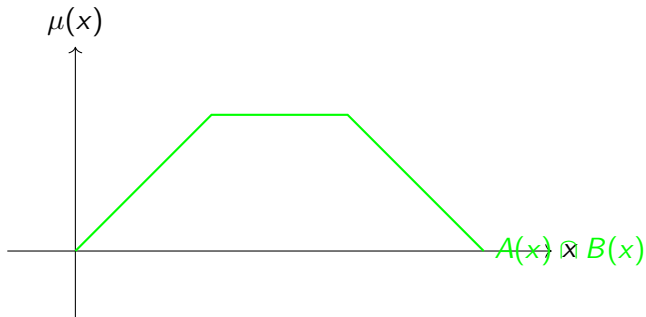


Gráfico dos Conjuntos Fuzzy

Gráfico de $A(x)$ e $B(x)$:

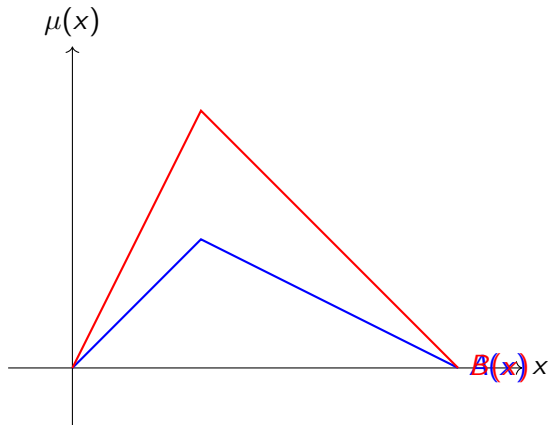
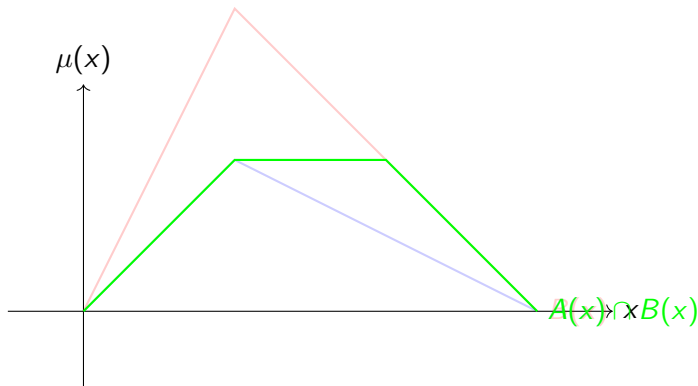


Gráfico da Interseção

Gráfico da função $\mu_{A \cap B}(x)$:



Inferência Fuzzy

Inferência Fuzzy e Regras Fuzzy

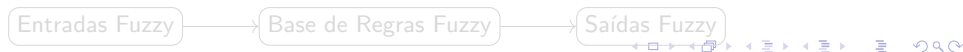
Inferência Fuzzy

O processo de inferência fuzzy é usado para derivar conclusões a partir de regras fuzzy aplicadas a dados de entrada imprecisos ou incertos.

Regras Fuzzy

Regras fuzzy são declaradas na forma de sentenças condicionais:

- ▶ **Forma geral:** *SE* antecedente *ENTÃO* consequente
- ▶ **Exemplo:** *SE temperatura é alta ENTÃO velocidade do ventilador é alta*
- ▶ Regras fuzzy utilizam variáveis linguísticas e suas funções de pertinência para determinar resultados.
- ▶ A agregação de regras permite combinar múltiplas condições.



Inferência Fuzzy e Regras Fuzzy

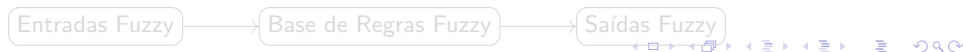
Inferência Fuzzy

O processo de inferência fuzzy é usado para derivar conclusões a partir de regras fuzzy aplicadas a dados de entrada imprecisos ou incertos.

Regras Fuzzy

Regras fuzzy são declaradas na forma de sentenças condicionais:

- ▶ **Forma geral:** *SE* antecedente *ENTÃO* consequente
- ▶ **Exemplo:** *SE temperatura é alta ENTÃO velocidade do ventilador é alta*
- ▶ Regras fuzzy utilizam variáveis linguísticas e suas funções de pertinência para determinar resultados.
- ▶ A agregação de regras permite combinar múltiplas condições.



Inferência Fuzzy e Regras Fuzzy

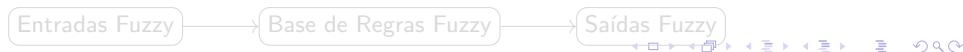
Inferência Fuzzy

O processo de inferência fuzzy é usado para derivar conclusões a partir de regras fuzzy aplicadas a dados de entrada imprecisos ou incertos.

Regras Fuzzy

Regras fuzzy são declaradas na forma de sentenças condicionais:

- ▶ **Forma geral:** *SE* antecedente *ENTÃO* consequente
- ▶ **Exemplo:** *SE temperatura é alta ENTÃO velocidade do ventilador é alta*
- ▶ Regras fuzzy utilizam variáveis linguísticas e suas funções de pertinência para determinar resultados.
- ▶ A agregação de regras permite combinar múltiplas condições.



Inferência Fuzzy e Regras Fuzzy

Inferência Fuzzy

O processo de inferência fuzzy é usado para derivar conclusões a partir de regras fuzzy aplicadas a dados de entrada imprecisos ou incertos.

Regras Fuzzy

Regras fuzzy são declaradas na forma de sentenças condicionais:

- ▶ **Forma geral:** *SE* antecedente *ENTÃO* consequente
- ▶ **Exemplo:** *SE temperatura é alta ENTÃO velocidade do ventilador é alta*
- ▶ Regras fuzzy utilizam variáveis linguísticas e suas funções de pertinência para determinar resultados.
- ▶ A agregação de regras permite combinar múltiplas condições.



Sistemas de Inferência Fuzzy

Método de Mamdani

- ▶ Um dos métodos mais usados para inferência fuzzy.
- ▶ As saídas fuzzy são obtidas por um processo de agregação e defuzzificação.
- ▶ Adequado para sistemas onde as regras são interpretáveis e compreensíveis.

Exemplo de Regras de Mamdani

- ▶ SE temperatura é alta ENTÃO ventilador é rápido.
- ▶ SE temperatura é baixa ENTÃO ventilador é lento.

Método de Sugeno

Características principais

- ▶ Baseado em expressões matemáticas ou funções lineares.
- ▶ A saída fuzzy é convertida diretamente em um valor numérico, sem defuzzificação.
- ▶ Adequado para sistemas complexos e onde é necessário controle preditivo.

Exemplo de Regras de Sugeno

- ▶ SE temperatura é alta ENTÃO $velocidade = 5 + 2temp$
- ▶ SE temperatura é baixa ENTÃO $velocidade = 1 + 0.5temp$

Comparação entre Mamdani e Sugeno

Diferenças principais

- ▶ **Mamdani:**
 - ▶ Foco em interpretabilidade.
 - ▶ Saída é uma função fuzzy que requer defuzzificação.
- ▶ **Sugeno:**
 - ▶ Foco em precisão e simplicidade computacional.
 - ▶ Saída é numérica, sem necessidade de defuzzificação.

Aplicações

- ▶ **Mamdani:** Sistemas de suporte à decisão, controle de dispositivos como ventiladores e condicionadores de ar.
- ▶ **Sugeno:** Controle avançado de sistemas complexos, como robótica e automação industrial.

Agregação

- ▶ Combina as saídas fuzzy de todas as regras aplicáveis para formar um único conjunto fuzzy.
- ▶ Métodos comuns de agregação:
 - ▶ União: $\mu_{agregado}(x) = \max(\mu_{regra1}(x), \mu_{regra2}(x), \dots)$
 - ▶ Soma: $\mu_{agregado}(x) = \sum \mu_{regra_i}(x)$

Defuzzificação

- ▶ Converte o conjunto fuzzy agregado em um valor numérico para uso no mundo real.
- ▶ Métodos comuns de defuzzificação:
 - ▶ Centro de Gravidade (CoG): $y = \frac{\int x \cdot \mu(x) dx}{\int \mu(x) dx}$
 - ▶ Média dos Máximos (MoM): $y = \text{média}(x \text{ onde } \mu(x) \text{ é máximo})$
 - ▶ Altura: $y = \sum \mu(x_i) \cdot x_i / \sum \mu(x_i)$

Agregação

- ▶ Combina as saídas fuzzy de todas as regras aplicáveis para formar um único conjunto fuzzy.
- ▶ Métodos comuns de agregação:
 - ▶ União: $\mu_{agregado}(x) = \max(\mu_{regra1}(x), \mu_{regra2}(x), \dots)$
 - ▶ Soma: $\mu_{agregado}(x) = \sum \mu_{regra_i}(x)$

Defuzzificação

- ▶ Converte o conjunto fuzzy agregado em um valor numérico para uso no mundo real.
- ▶ Métodos comuns de defuzzificação:
 - ▶ Centro de Gravidade (CoG): $y = \frac{\int x \cdot \mu(x) dx}{\int \mu(x) dx}$
 - ▶ Média dos Máximos (MoM): $y = \text{média}(x \text{ onde } \mu(x) \text{ é máximo})$
 - ▶ Altura: $y = \sum \mu(x_i) \cdot x_i / \sum \mu(x_i)$

Sistemas de Controle Fuzzy

Controladores Fuzzy

Definição

Um controlador fuzzy é um sistema que utiliza lógica fuzzy para determinar as ações de controle baseadas em entradas incertas ou imprecisas.

Componentes de um Controlador Fuzzy

▶ Fuzzificação:

- ▶ Transforma as entradas nítidas (crisp) em valores fuzzy.
- ▶ Usa funções de pertinência para determinar o grau de pertencimento das entradas aos conjuntos fuzzy definidos.

▶ Inferência:

- ▶ Aplica regras fuzzy para avaliar as condições e determinar as saídas fuzzy correspondentes.
- ▶ Utiliza operadores lógicos fuzzy (e.g., AND, OR) para combinar as regras.

Componentes de um Controlador Fuzzy

▶ Defuzzificação:

- ▶ Converte as saídas fuzzy em um valor nítido (crisp) para aplicação no sistema controlado.
- ▶ Métodos comuns incluem:
 - ▶ Centro de Gravidade (CoG): Proporciona equilíbrio considerando toda a área do conjunto fuzzy agregado.
 - ▶ Média dos Máximos (MoM): Usa o valor médio dos pontos com grau de pertinência máximo.

Componentes de um Controlador Fuzzy

Exemplo de Aplicação

- ▶ Controle de temperatura: SE temperatura é alta ENTÃO potência do ar-condicionado é alta.
- ▶ Controle de velocidade: SE distância do carro da frente é pequena ENTÃO velocidade é baixa.

Aprofunde-se

Consulte sempre o Material Didático

Os slides são um guia didático. Para teoria completa, fórmulas e demonstrações, acesse o **Módulo 5** e os **notebooks Colab**.

- ▶ **Módulo 5:** Redes Neurais, Deep Learning, Computação Evolutiva, Fuzzy, AutoML
- ▶ **Colab:** logicaFuzzy.ipynb
- ▶  <https://github.com/eltonsarmanho>

Redes Neurais

Objetivos da Apresentação

- ▶ Explorar as características fundamentais das Redes Neurais Artificiais (RNAs).
- ▶ Compreender o funcionamento básico de uma RNA.
- ▶ Analisar as principais arquiteturas de RNAs disponíveis.
- ▶ Ilustrar aplicações de RNAs em Reconhecimento de Padrões e Controle.

Motivação Biológica

Ideia Central

- ▶ Utilizar neurônios biológicos como modelos para neurônios artificiais.
- ▶ Neurônios biológicos são o elemento fundamental do sistema nervoso.
- ▶ Existem diversos tipos de neurônios biológicos.

Neurônio Biológico

- ▶ O cérebro humano é relativamente lento, mas possui alto paralelismo.
- ▶ Cerca de 10^{11} neurônios, cada um operando a aproximadamente 1 KHz.
- ▶ Cada neurônio pode se conectar com até 10^4 outros neurônios.

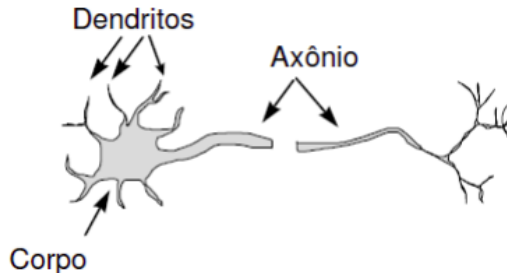


Figura: Representação de um neurônio biológico.

Funcionamento Simplificado de um Neurônio Biológico

- ▶ Neurônios podem estar em dois estados:
 - ▶ **Ativo ou excitado:** Envia sinais para outros neurônios por meio do axônio e sinapses.
 - ▶ **Inativo ou inibido:** Não envia sinais.
- ▶ Sinapses podem ser de dois tipos:
 - ▶ **Excitatorias:** Excitam o neurônio receptor.
 - ▶ **Inibitórias:** Inibem o neurônio receptor.

Ativação do Neurônio

- ▶ Quando o efeito cumulativo das várias sinapses que chegam a um neurônio excede um valor limite, o neurônio dispara.
- ▶ O neurônio fica ativo por um período e envia um sinal para outros neurônios.

Introdução às Redes Neurais Artificiais

- ▶ Redes neurais artificiais são modelos computacionais inspirados no funcionamento do cérebro humano.
- ▶ Esses modelos são amplamente utilizados em aprendizado de máquina e inteligência artificial.
- ▶ As redes neurais foram desenvolvidas a partir de contribuições de diversas áreas, como matemática aplicada, estatística e ciência da computação.

Origens das Redes Neurais Artificiais

- ▶ Os primeiros estudos formais sobre redes neurais artificiais foram realizados por McCulloch e Pitts em 1943.
- ▶ Eles propuseram um modelo matemático para um neurônio artificial.
- ▶ Esse modelo era capaz de implementar operações lógicas básicas, como "E" e "OU".

Modelo Matemático do Neurônio Artificial

- ▶ Um neurônio artificial pode ser representado por uma função $\eta : \mathbb{R}^n \rightarrow \{0, 1\}$.
- ▶ A função é definida pela equação:

$$\eta(x) = \chi_{\geq 0} \left[w^T x - b \right],$$

onde:

- ▶ w é o vetor de pesos,
- ▶ x é o vetor de entradas,
- ▶ b é o limiar de ativação,
- ▶ $\chi_{\geq 0}$ é a função indicadora que retorna 1 se o argumento for maior ou igual a zero, e 0 caso contrário.

Aplicações Iniciais

- ▶ O modelo de McCulloch e Pitts demonstrou que redes neurais podem resolver problemas de lógica clássica.
- ▶ Essa descoberta foi um marco inicial para o desenvolvimento de sistemas de inteligência artificial.
- ▶ Hoje, as redes neurais são aplicadas em diversas áreas, como reconhecimento de padrões, processamento de linguagem natural e visão computacional.

Neurônio Artificial

- ▶ Um neurônio artificial é uma função $\eta : \mathbb{R}^n \rightarrow \mathbb{R}$.
- ▶ A função é definida por:

$$\eta(x) = \varphi(w^T x - b),$$

onde:

- ▶ φ é a função de ativação,
- ▶ w é o vetor de pesos sinápticos,
- ▶ b é o viés.

- ▶ A função de ativação φ determina a saída do neurônio.

Função Limiar

$$\chi_{\geq 0}(t) = \begin{cases} 1, & t \geq 0, \\ 0, & t < 0. \end{cases}$$

Função Logística

$$\sigma(t) = \frac{1}{(1 + e^{-t})}.$$

Função ReLU

$$\text{relu}(t) = \max\{0, t\}.$$

Aplicações das Funções de Ativação

- ▶ A função limiar é usada em modelos binários.
- ▶ A função logística é comum em problemas de classificação.
- ▶ A função ReLU é amplamente utilizada em redes neurais profundas.

Propriedades das Redes Neurais Artificiais

- ▶ As redes neurais artificiais (RNAs) possuem características que as tornam poderosas para diversas aplicações.
- ▶ Entre as principais propriedades estão: aprendizado, generalização e abstração.

Aprendizado

- ▶ As RNAs são capazes de modificar seu comportamento com base nos dados de entrada.
- ▶ Isso permite que elas produzam saídas consistentes e adaptadas ao domínio do problema.
- ▶ O aprendizado ocorre através da ajuste dos pesos sinápticos durante o treinamento.

Generalização

- ▶ Após o treinamento, uma RNA pode lidar com pequenas variações nas entradas, como ruídos ou distorções.
- ▶ Essa capacidade de generalização é essencial para aplicações em cenários do mundo real.
- ▶ A generalização permite que a rede mantenha um bom desempenho mesmo com dados não vistos durante o treinamento.

Arquitetura de Redes Neurais

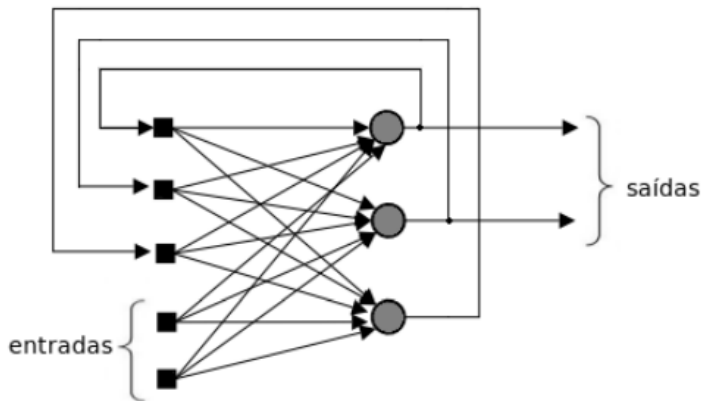
- ▶ Os neurônios artificiais são as unidades básicas de processamento em uma rede neural.
- ▶ Uma rede neural pode ser representada como um grafo direcionado.
- ▶ Nesse grafo, os vértices correspondem aos neurônios e as arestas indicam conexões entre eles.

Topologia da Rede Neural

- ▶ A estrutura do grafo é chamada de arquitetura ou topologia da rede neural.
- ▶ A topologia define como os neurônios estão organizados e conectados.
- ▶ Existem diferentes tipos de topologias, como redes neurais recorrentes e progressivas.

Redes Neurais Recorrentes ou *Feed-backward networks*

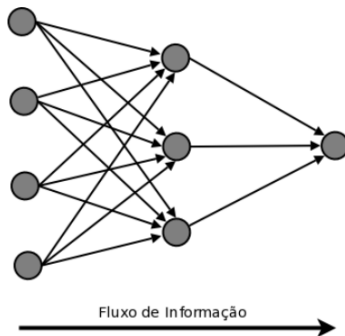
- ▶ Uma rede neural é dita recorrente quando o grafo associado contém ciclos.
- ▶ Isso permite que a informação flua em loops, o que é útil para modelar sequências temporais.
- ▶ Aplicações comuns incluem processamento de linguagem natural e previsão de séries temporais.

Redes Neurais Recorrentes ou *Feed-backward networks*

Redes Neurais Progressivas ou *Feed-forward networks*

- ▶ Uma rede neural é progressiva quando o fluxo de informação segue em uma única direção, sem ciclos.
- ▶ Também conhecidas como redes feedforward, são amplamente utilizadas em tarefas de classificação e regressão.
- ▶ Exemplos incluem redes neurais convolucionais (CNNs) e perceptrons multicamadas.

Redes Neurais Progressivas ou *Feed-forward networks*



Modelo Matemático do Perceptron

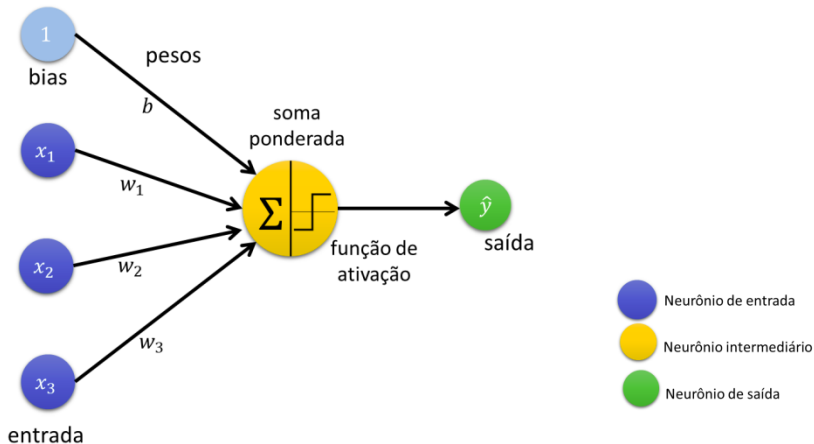
- ▶ O Perceptron é uma função $f : \mathbb{R}^n \rightarrow \{0, 1\}$ definida por:

$$f(x) = \begin{cases} 1, & \text{se } w^T x + b \geq 0, \\ 0, & \text{caso contrário,} \end{cases}$$

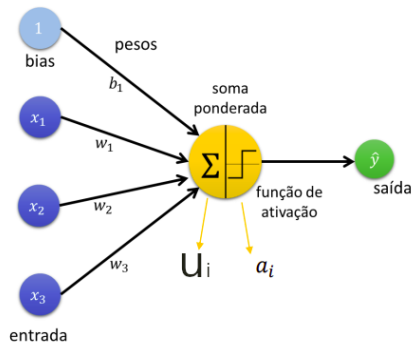
onde:

- ▶ $x \in \mathbb{R}^n$ é o vetor de entrada,
- ▶ $w \in \mathbb{R}^n$ é o vetor de pesos,
- ▶ $b \in \mathbb{R}$ é o viés.

Estrutura do Perceptron



Estrutura do Perceptron



Vetor de entrada, de dimensão $(1, n_x)$

$$\mathbf{x}^{(i)} = [x_1, x_2, \dots, x_{n_x}]$$

Vetor de pesos, de dimensão $(1, n_x)$

$$\mathbf{w} = [w_1, w_2, \dots, w_{n_x}]$$

Estado do neurônio, escalar

$$U_i = w_1 x_1 + \dots + w_{n_x} x_{n_x} + b = \sum_{j=0}^{n_x} x_j^{(i)} w_j + b = \mathbf{w}^T \mathbf{x}^{(i)} + b$$

Ativação, escalar

$$a_i = g(U_i)$$

Previsão, escalar

$$\hat{y}_i = a_i$$

Neurônio que Implementa a Porta OR

- ▶ Considere um neurônio com duas entradas x_1 e x_2 .
- ▶ Os pesos sinápticos são $w_1 = 1$ e $w_2 = 1$.
- ▶ O limiar de ativação é $\theta = 0,5$.
- ▶ A saída y é dada por:

$$y = \begin{cases} 1, & \text{se } u \geq 0, \\ 0, & \text{se } u < 0, \end{cases}$$

onde $u = w_1x_1 + w_2x_2 - \theta$.

Cálculo da Saída para a Porta OR

- ▶ A porta OR é definida pela seguinte tabela verdade:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

- ▶ Vamos calcular a saída y para cada combinação de entradas.

Exemplo de Cálculo

- Para $x_1 = 0$ e $x_2 = 0$:

$$u = (1 \cdot 0) + (1 \cdot 0) - 0,5 = -0,5 \Rightarrow y = 0.$$

- Para $x_1 = 0$ e $x_2 = 1$:

$$u = (1 \cdot 0) + (1 \cdot 1) - 0,5 = 0,5 \Rightarrow y = 1.$$

- Para $x_1 = 1$ e $x_2 = 0$:

$$u = (1 \cdot 1) + (1 \cdot 0) - 0,5 = 0,5 \Rightarrow y = 1.$$

- Para $x_1 = 1$ e $x_2 = 1$:

$$u = (1 \cdot 1) + (1 \cdot 1) - 0,5 = 1,5 \Rightarrow y = 1.$$

Regra de Treinamento do Perceptron

- ▶ A regra de treinamento do Perceptron ajusta os pesos w e o viés b para classificar corretamente os dados de treinamento.
- ▶ Para um conjunto de dados linearmente separável, o algoritmo converge em um número finito de passos.
- ▶ A atualização dos pesos é dada por:

$$w \leftarrow w + \eta(y_i - \hat{y}_i)x_i,$$

onde:

- ▶ η é a taxa de aprendizado,
- ▶ y_i é a saída desejada,
- ▶ $\hat{y}_i$ é a saída prevista.

Neurônio que Implementa a Porta AND

- ▶ Considere um neurônio com duas entradas x_1 e x_2 .
- ▶ Inicialmente, os pesos sinápticos são $w_1 = 0,2$ e $w_2 = 0,2$.
- ▶ O limiar de ativação é $\theta = 0,5$.
- ▶ A saída y é dada por:

$$y = \begin{cases} 1, & \text{se } u \geq 0, \\ 0, & \text{se } u < 0, \end{cases}$$

onde $u = w_1x_1 + w_2x_2 - \theta$.

Regra de Atualização dos Pesos

- ▶ A regra de atualização dos pesos é dada por:

$$w_i \leftarrow w_i + \eta(y_{\text{esperado}} - y_{\text{calculado}})x_i,$$

onde:

- ▶ η é a taxa de aprendizado (vamos usar $\eta = 0,1$),
- ▶ y_{esperado} é a saída desejada,
- ▶ $y_{\text{calculado}}$ é a saída calculada pelo neurônio.

Passo 1: $x_1 = 1, x_2 = 1$

- ▶ Saída esperada: $y_{\text{esperado}} = 1$.

- ▶ Cálculo de u :

$$u = (0,2 \cdot 1) + (0,2 \cdot 1) - 0,5 = -0,1 \Rightarrow y = 0.$$

- ▶ Erro: $y_{\text{esperado}} - y_{\text{calculado}} = 1 - 0 = 1$.

- ▶ Atualização dos pesos:

$$w_1 \leftarrow 0,2 + 0,1 \cdot (1 - 0) \cdot 1 = 0,3,$$

$$w_2 \leftarrow 0,2 + 0,1 \cdot (1 - 0) \cdot 1 = 0,3.$$

- ▶ Novos pesos: $w_1 = 0,3, w_2 = 0,3$.

Passo 1.1: $x_1 = 1, x_2 = 1$

► Saída esperada: $y_{\text{esperado}} = 1$.

► Cálculo de u :

$$u = (0,3 \cdot 1) + (0,3 \cdot 1) - 0,5 = 0,1 \Rightarrow y = 1.$$

► Erro: $y_{\text{esperado}} - y_{\text{calculado}} = 1 - 1 = 0$.

► Não há atualização dos pesos.

Passo 2: $x_1 = 1, x_2 = 0$

▶ Saída esperada: $y_{\text{esperado}} = 0$.

▶ Cálculo de u :

$$u = (0,3 \cdot 1) + (0,3 \cdot 0) - 0,5 = -0,2 \Rightarrow y = 0.$$

▶ Erro: $y_{\text{esperado}} - y_{\text{calculado}} = 0 - 0 = 0$.

▶ Não há atualização dos pesos.

Passo 3: $x_1 = 0, x_2 = 1$

▶ Saída esperada: $y_{\text{esperado}} = 0$.

▶ Cálculo de u :

$$u = (0,3 \cdot 0) + (0,3 \cdot 1) - 0,5 = -0,2 \Rightarrow y = 0.$$

▶ Erro: $y_{\text{esperado}} - y_{\text{calculado}} = 0 - 0 = 0$.

▶ Não há atualização dos pesos.

Passo 4: $x_1 = 0, x_2 = 0$

► Saída esperada: $y_{\text{esperado}} = 0$.

► Cálculo de u :

$$u = (0,3 \cdot 0) + (0,3 \cdot 0) - 0,5 = -0,5 \Rightarrow y = 0.$$

► Erro: $y_{\text{esperado}} - y_{\text{calculado}} = 0 - 0 = 0$.

► Não há atualização dos pesos.

Função de Ativação

- ▶ A função de ativação $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ é aplicada à saída de um neurônio artificial.
- ▶ Ela introduz não linearidade no modelo, permitindo que a rede neural aprenda padrões complexos.
- ▶ Sem a não linearidade, uma rede neural com múltiplas camadas seria equivalente a uma única transformação linear.

Importância da Não Linearidade

- ▶ Em redes neurais com várias camadas ocultas, a ausência de não linearidade resultaria em uma composição de transformações afins.
- ▶ Matematicamente, isso pode ser expresso como:

$$f(x) = W_n(W_{n-1}(\dots W_1(x) + b_1) + b_{n-1}) + b_n,$$

onde W_i são matrizes de pesos e b_i são vetores de viés.

- ▶ Sem funções de ativação não lineares, $f(x)$ seria equivalente a uma única transformação afim:

$$f(x) = W'x + b'.$$

Exemplos de Funções de Ativação

- ▶ **Função Limiar (Step):**

$$\varphi(t) = \begin{cases} 1, & t \geq 0, \\ 0, & t < 0. \end{cases}$$

- ▶ **Função Logística (Sigmoid):**

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

- ▶ **Função ReLU (Rectified Linear Unit):**

$$\text{ReLU}(t) = \max(0, t).$$

Impacto na Aprendizagem

- ▶ A não linearidade introduzida pela função de ativação permite que a rede neural modele relações complexas entre entradas e saídas.
- ▶ Sem funções de ativação não lineares, a rede não seria capaz de aprender funções não lineares, limitando sua aplicabilidade.
- ▶ A escolha da função de ativação afeta a eficiência do treinamento e a capacidade de generalização do modelo.

Limitações do Perceptron

- ▶ O Perceptron só pode classificar dados linearmente separáveis.
- ▶ Ele não consegue resolver problemas não lineares, como o problema do XOR (ou-exclusivo).
- ▶ Essa limitação foi destacada por Minsky e Papert em 1969, o que levou a um declínio temporário no interesse por redes neurais.

Definição Matemática de uma RNA

- ▶ Uma Rede Neural Artificial (RNA) pode ser vista como uma função f que mapeia um conjunto de dados de entrada X para um conjunto de saídas Y .
- ▶ Formalmente, a RNA é uma função:

$$f : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_y},$$

onde:

- ▶ n_x é o número de features (entradas),
- ▶ n_y é o número de saídas.

Representação dos Dados

- ▶ Os dados de entrada são representados por uma matriz $X \in \mathbb{R}^{n_x \times m}$, onde:
 - ▶ n_x é o número de features por exemplo,
 - ▶ m é o número total de exemplos de treinamento.
- ▶ As saídas desejadas são representadas por uma matriz $Y \in \mathbb{R}^{n_y \times m}$, onde:
 - ▶ n_y é o número de saídas por exemplo.

Processo de Aprendizagem

- ▶ A principal característica de uma rede neural é sua capacidade de aprender e melhorar o desempenho com base em estímulos externos.
- ▶ A aprendizagem envolve a atualização da representação interna da rede, ajustando sua arquitetura e os pesos das conexões entre neurônios.
- ▶ Esse processo permite que a rede desempenhe tarefas específicas de forma mais eficiente ao longo do tempo.

Atualização da Representação Interna

- ▶ A aprendizagem ocorre por meio da modificação da arquitetura da rede e do ajuste dos pesos sinápticos.
- ▶ As regras de aprendizagem determinam como os pesos são atualizados em resposta aos erros ou estímulos externos.
- ▶ Essas regras são essenciais para garantir que a rede se adapte e melhore seu desempenho.

Tipos de Regras de Aprendizagem

- ▶ **Aprendizagem por Correção de Erro:**
 - ▶ Utilizada em treinamento supervisionado.
 - ▶ Os pesos são ajustados com base no erro, que é a diferença entre a saída da rede e o valor esperado.
 - ▶ O erro é minimizado gradualmente ao longo dos ciclos de treinamento.
- ▶ Outras regras de aprendizagem incluem:
 - ▶ Aprendizagem Hebbiana,
 - ▶ Aprendizagem Competitiva,
 - ▶ Aprendizagem por Reforço.

Aprendizagem por Correção de Erro

- ▶ A regra de correção de erro é uma das mais comuns em redes neurais supervisionadas.
- ▶ O erro é calculado como:

$$\text{Erro} = y_{\text{esperado}} - y_{\text{calculado}}.$$

- ▶ Os pesos são atualizados usando a fórmula:

$$w_i \leftarrow w_i + \eta \cdot \text{Erro} \cdot x_i,$$

onde η é a taxa de aprendizado e x_i é a entrada.

Treinamento da RNA

- ▶ Para que a RNA aprenda a mapear X para Y , ela precisa ser treinada.
- ▶ O treinamento é um processo de otimização que minimiza uma função de perda $\mathcal{L}$, que mede a diferença entre as saídas previstas $\hat{Y}$ e as saídas desejadas Y :

$$\mathcal{L}(Y, \hat{Y}) = \frac{1}{m} \sum_{i=1}^m \|y_i - \hat{y}_i\|^2.$$

- ▶ Durante o treinamento, os parâmetros da RNA (pesos e vieses) são ajustados para minimizar $\mathcal{L}$.

Aprendizado Supervisionado

- ▶ O treinamento de uma RNA é tipicamente supervisionado, o que requer um conjunto de dados rotulados $\{(x_i, y_i)\}_{i=1}^m$.
- ▶ A cada iteração, a RNA gera uma saída $\hat{y}_i$ para a entrada x_i .
- ▶ A diferença entre y_i e $\hat{y}_i$ é usada para atualizar os parâmetros da rede via retropropagação.

Introdução ao Multi-Layer Perceptron (MLP)

- ▶ As limitações do Perceptron simples levaram ao desenvolvimento de redes neurais com múltiplas camadas, como o Adaline e o MLP.
- ▶ O MLP (Multi-Layer Perceptron) foi proposto por Rumelhart e McClelland em 1986, com base em trabalhos anteriores de Widrow e Hoff (1960).
- ▶ O MLP é composto por várias camadas de neurônios artificiais, permitindo a modelagem de funções não lineares complexas.

Introdução ao MLP

- ▶ O Multi-Layer Perceptron (MLP) é uma rede neural artificial com múltiplas camadas de neurônios.
- ▶ Ele é composto por:
 - ▶ Uma camada de entrada,
 - ▶ Uma ou mais camadas ocultas,
 - ▶ Uma camada de saída.
- ▶ A topologia do MLP permite a modelagem de funções não lineares complexas.

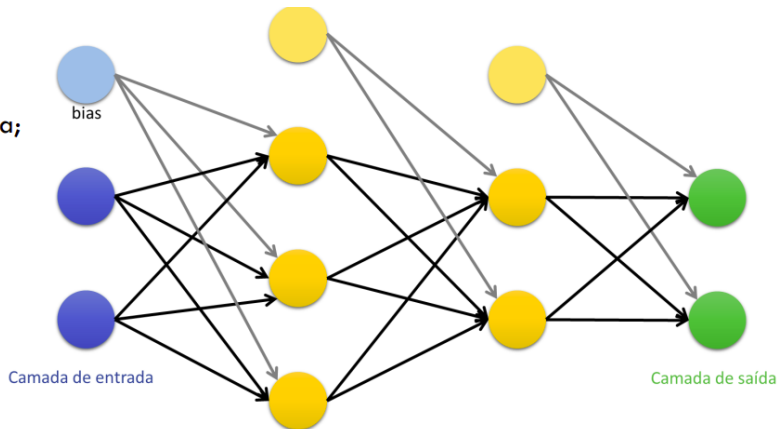
Topologia do MLP

- ▶ A topologia de um MLP é definida pela organização das camadas e pelas conexões entre os neurônios.
- ▶ Cada camada é composta por um conjunto de neurônios que processam informações e passam os resultados para a camada seguinte.
- ▶ As conexões entre os neurônios são representadas por pesos sinápticos.

Topologia do MLP

- Entrada;
- Intermediária ou escondida;
- Saída.

Exemplo de RNA com $L = 3$ camadas intermediárias (veja que a camada de saída entra na conta)



Definição Matemática de uma Camada

- ▶ Uma camada de m neurônios pode ser representada por uma função $L : \mathbb{R}^n \rightarrow \mathbb{R}^m$.
- ▶ A função de uma camada é dada por:

$$L(x) = \varphi(Wx - b),$$

onde:

- ▶ $W \in \mathbb{R}^{m \times n}$ é a matriz de pesos sinápticos,
- ▶ $b \in \mathbb{R}^m$ é o vetor de vieses,
- ▶ φ é a função de ativação aplicada componente a componente.

Camada Densa ou Totalmente Conectada

- ▶ Uma camada é chamada de **densa** ou **totalmente conectada** se a matriz W é uma matriz cheia, ou seja, todos os elementos da matriz podem ser diferentes de zero.
- ▶ Isso significa que cada neurônio na camada recebe entradas de todos os neurônios da camada anterior.
- ▶ Matematicamente, a densidade da camada é expressa pela estrutura completa da matriz W .

Exemplo de MLP com Duas Camadas

- ▶ Considere um MLP com duas camadas:
 - ▶ A primeira camada $L_1 : \mathbb{R}^n \rightarrow \mathbb{R}^k$ com função de ativação φ_1 .
 - ▶ A segunda camada $L_2 : \mathbb{R}^k \rightarrow \mathbb{R}^m$ com função de ativação φ_2 .
- ▶ A função total do MLP é dada por:

$$f(x) = L_2(L_1(x)) = \varphi_2(W_2\varphi_1(W_1x - b_1) - b_2).$$

- ▶ Essa composição de camadas permite a modelagem de funções não lineares complexas.

Papel das Camadas

- ▶ As primeiras camadas ($L_1, L_2, \dots$) atuam como **extratores de características**, transformando a entrada em representações intermediárias.
- ▶ As últimas camadas (L_{K-1}, L_K) realizam a tarefa de aprendizado de máquina, como classificação ou regressão.
- ▶ A camada L_0 pode ser incluída como a identidade, representando a entrada original.

Redes Neurais Profundas

- ▶ Uma rede neural é considerada profunda (**deep**) quando possui três ou mais camadas ($K \geq 3$).
- ▶ Redes profundas são capazes de modelar funções altamente não lineares e capturar relações complexas nos dados.
- ▶ A profundidade da rede permite a extração de características hierárquicas, onde camadas iniciais detectam padrões simples e camadas posteriores combinam esses padrões em representações mais complexas.

Exemplo de Rede Profunda

- ▶ Considere uma rede com $K = 4$ camadas:

- ▶ $L_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{h_1},$
- ▶ $L_2 : \mathbb{R}^{h_1} \rightarrow \mathbb{R}^{h_2},$
- ▶ $L_3 : \mathbb{R}^{h_2} \rightarrow \mathbb{R}^{h_3},$
- ▶ $L_4 : \mathbb{R}^{h_3} \rightarrow \mathbb{R}^m.$

- ▶ A função total da rede é:

$$N(x) = L_4(L_3(L_2(L_1(x)))).$$

- ▶ Cada camada aplica uma transformação não linear, permitindo que a rede aprenda representações complexas.

Treinamento de Redes Neurais

- ▶ O treinamento de uma rede neural (superficial ou profunda) envolve a minimização de uma função de perda.
- ▶ Em problemas de regressão, a função de perda comum é o **Erro Quadrático Médio (MSE)**:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2,$$

onde y_i é o valor esperado e $\hat{y}_i$ é a saída da rede.

- ▶ Em problemas de classificação, a função de perda comum é a **Entropia Cruzada**:

$$\text{Entropia Cruzada} = - \sum_{i=1}^m y_i \log(\hat{y}_i).$$

Formulação do Problema de Otimização

- ▶ O treinamento é formulado como um problema de otimização irrestrito nos parâmetros da rede (pesos W e vieses b).
- ▶ O objetivo é encontrar os parâmetros que minimizam a função de perda:

$$\min_{W, b} \mathcal{L}(W, b),$$

onde $\mathcal{L}$ é a função de perda.

- ▶ Devido ao grande número de parâmetros, métodos baseados no gradiente são utilizados.

Métodos Baseados no Gradiente

- ▶ O gradiente da função de perda em relação aos parâmetros é calculado usando a **regra da cadeia**.
- ▶ O gradiente é dado por:

$$\nabla_{W,b}\mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial W}, \frac{\partial \mathcal{L}}{\partial b} \right).$$

- ▶ O cálculo do gradiente é realizado de forma eficiente usando o algoritmo de retropropagação (*backpropagation*).

Algoritmo de Retropropagação

- ▶ O algoritmo de retropropagação consiste em duas fases:
 - ▶ **Fase Forward:** Calcula a saída da rede para uma dada entrada.
 - ▶ **Fase Backward:** Propaga o erro da saída para as camadas anteriores, calculando os gradientes.
- ▶ O gradiente é usado para atualizar os pesos e vieses:

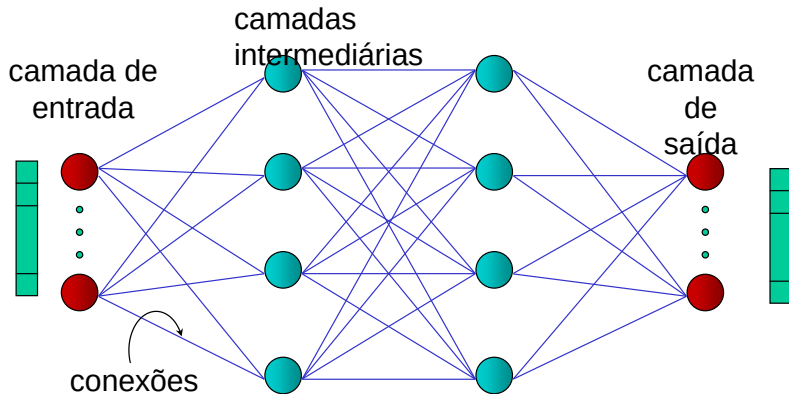
$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b},$$

onde η é a taxa de aprendizado.

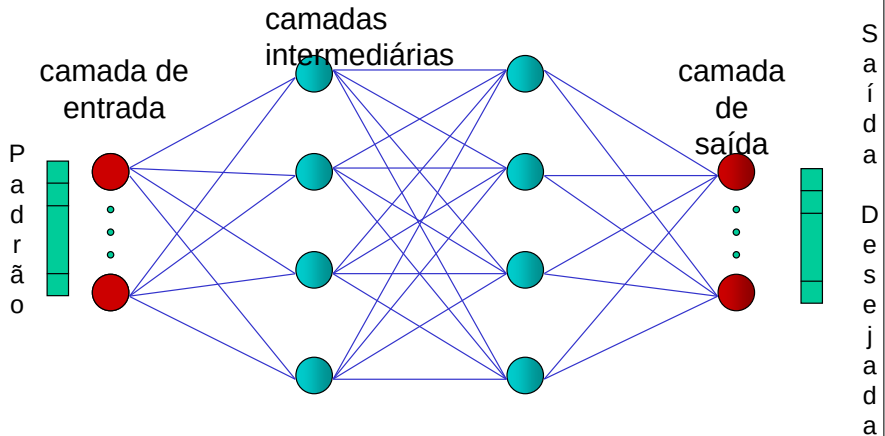
Ilustração do Algoritmo de Retropropagação

Veja o GIF animado:

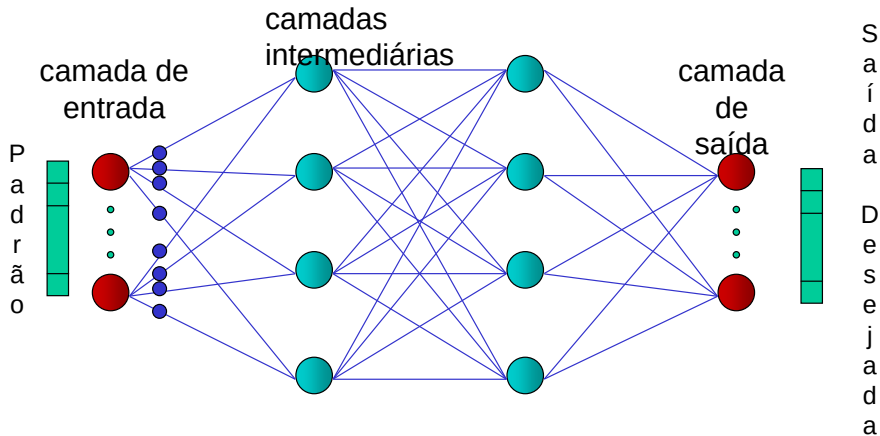
Rede MLP



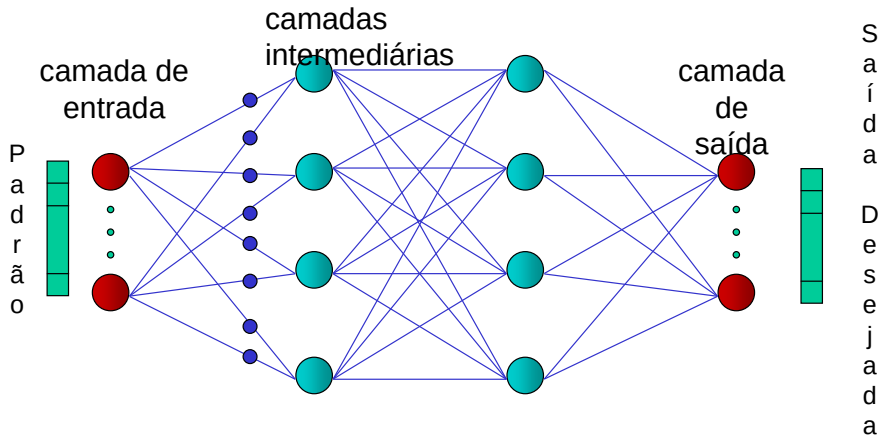
Aprendizado



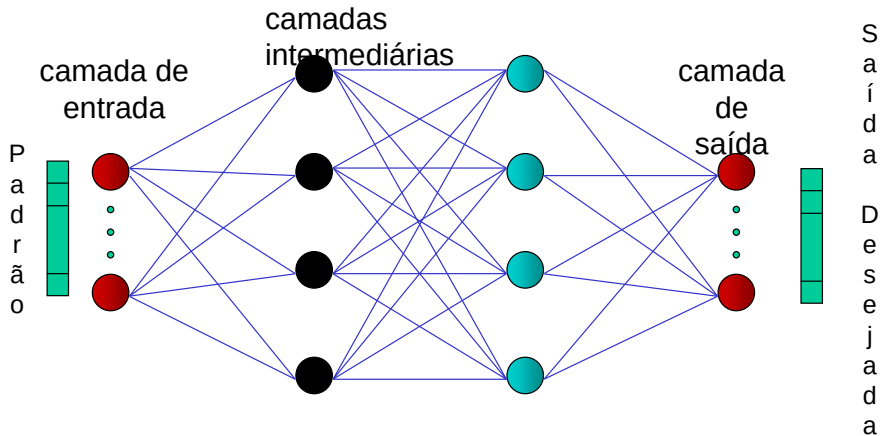
RNA - Aprendizado



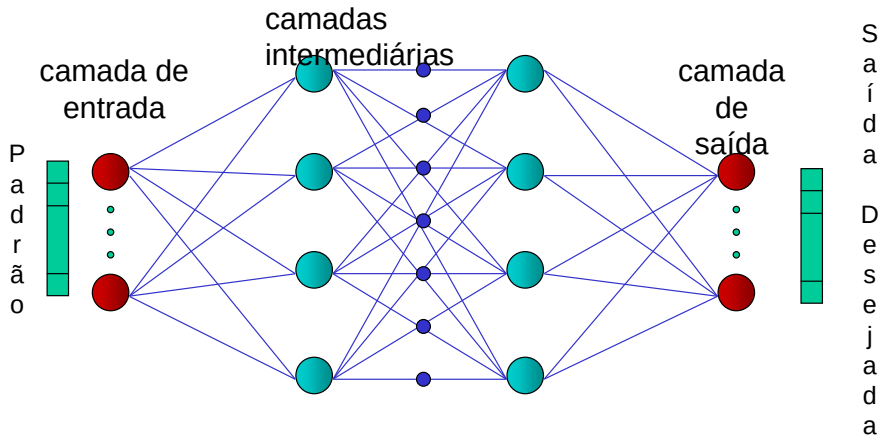
RNA - Aprendizado



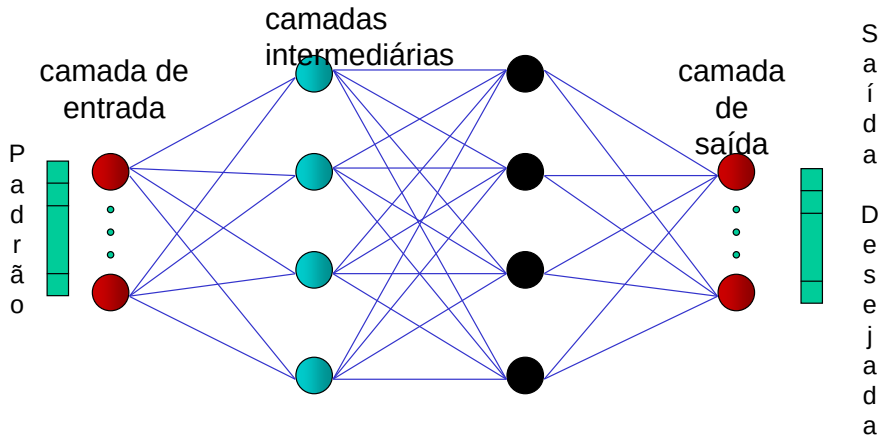
RNA - Aprendizado



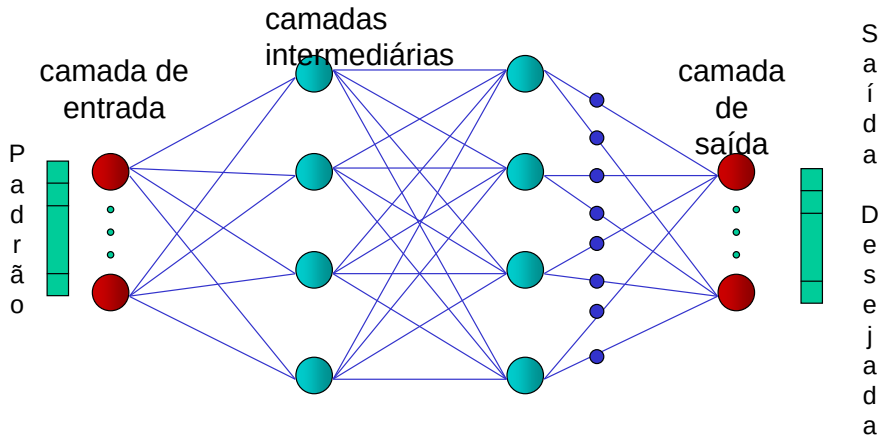
RNA - Aprendizado



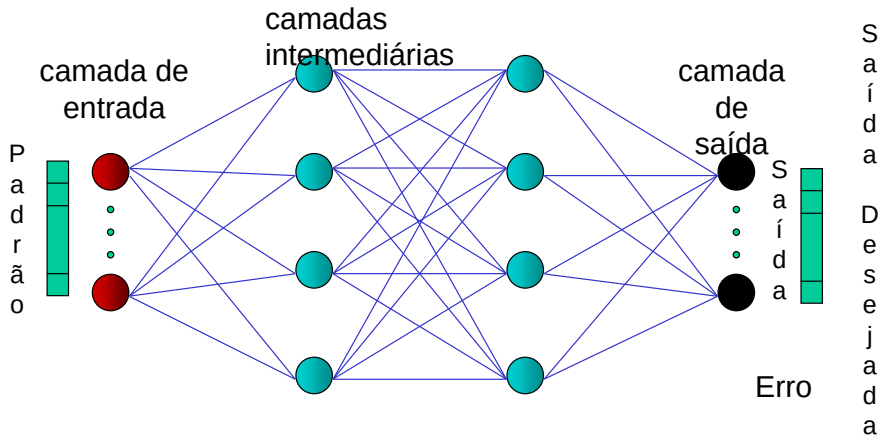
RNA - Aprendizado



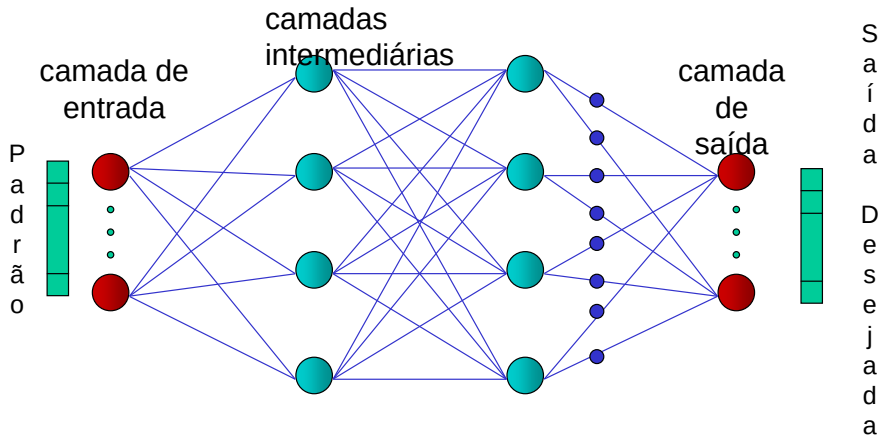
RNA - Aprendizado



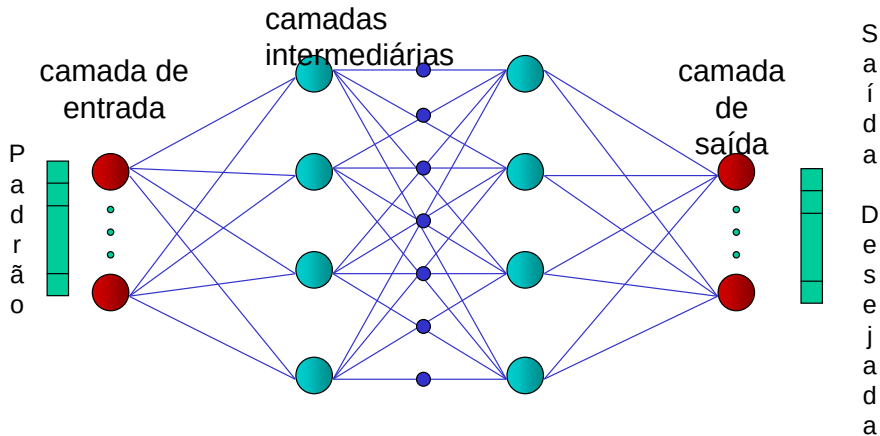
RNA - Aprendizado



RNA - Aprendizado



RNA - Aprendizado



RNA - Aprendizado

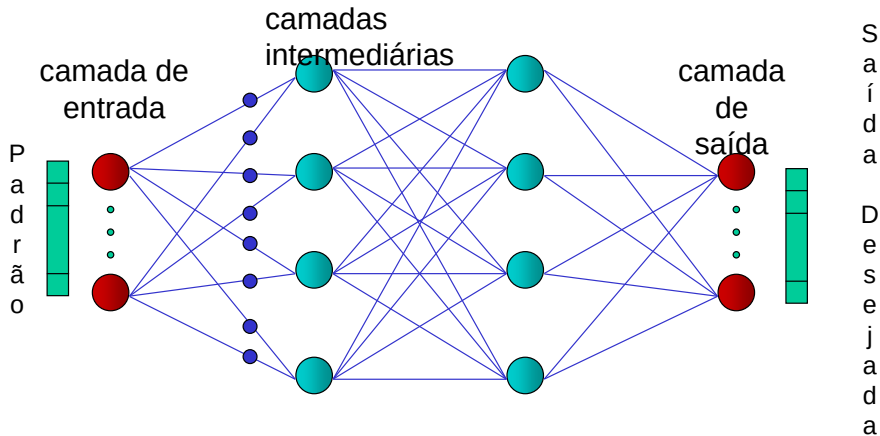
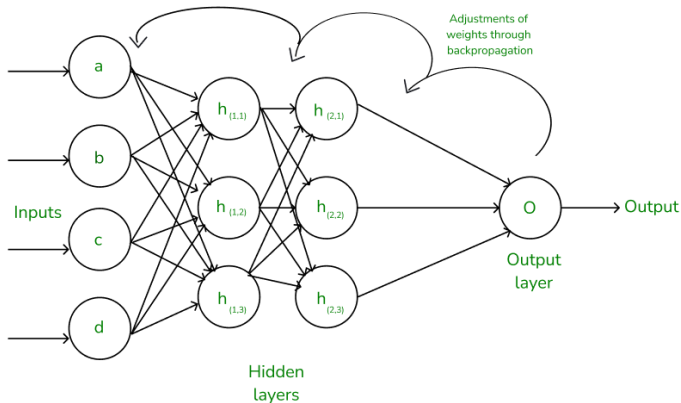


Ilustração do Algoritmo de Retropropagação

- A figura abaixo ilustra o fluxo de informações no algoritmo de retropropagação:



Em resumo

- ▶ O treinamento de redes neurais é um problema de otimização que busca minimizar uma função de perda.
- ▶ Métodos baseados no gradiente, como o algoritmo de retropropagação, são essenciais para o treinamento eficiente de redes neurais.
- ▶ A escolha da função de perda e da taxa de aprendizado é crucial para o desempenho da rede.

Redes Convolucionais (CNN)

Convolução

Filtro deslizante extrai **features locais**

$$(f * g)[n] = \sum_k f[k] g[n - k]$$

Pooling

Reduz dimensionalidade e adiciona **invariância** a translações

Max Pooling / Average Pooling

Aplicações em Mineração de Dados

- ▶ **Imagens:** extração automática de features visuais
- ▶ **Conv1D:** análise de séries temporais e sinais
- ▶ **Texto:** CNNs para classificação de documentos
- ▶ **Grafos:** Graph CNN para dados relacionais

➔ Consulte o Colab para exemplo prático com TensorFlow/Keras

Redes Recorrentes: RNN, LSTM e GRU

RNN — Recurrent Neural Network

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

Processa **sequências** com memória de estados anteriores

LSTM (Hochreiter & Schmidhuber, 1997)

Portões **forget** / **input** / **output** controlam o fluxo de informação

Evita o problema de *vanishing gradient*

GRU (Cho et al., 2014)

Versão simplificada da LSTM com portões **reset** e **update**

Menos parâmetros, performance similar

Aplicações em Mineração de Dados

- ▶ **Séries temporais:** previsão de demanda, clima
- ▶ **NLP:** análise de sentimento, tradução
- ▶ **Anomalias:** detecção em logs e eventos
- ▶ **Recomendação:** modelagem de sequências de itens

Transformers e Mecanismo de Atenção

Atenção Escalada (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Permite que cada posição “atenda” a todas as outras

BERT

Embeddings de texto de alta qualidade via pré-treino bidirecional

TabNet (Arik & Pfister, 2021)

- ▶ Transformer adaptado para **dados tabulares**
- ▶ Seleção de features **interpretável** passo a passo
- ▶ Atenção esparsa: foca nas features mais relevantes
- ▶ Competitivo com GBDTs em benchmarks

Ver Material Didático Módulo 5 para detalhes matemáticos

HPO: Otimização de Hiperparâmetros

Grid Search

$\mathcal{O}(n^k)$, método exaustivo e impraticável em alta dimensão

Testa todas as combinações possíveis

Random Search (Bergstra & Bengio, 2012)

Melhor cobertura do espaço de busca

Encontra boas soluções com menos avaliações

Otimização Bayesiana (TPE)

Usa modelo *surrogate* para balancear

exploração × **exploração**

Tree-structured Parzen Estimator

Optuna (Akiba et al., 2019)

Framework Python com:

- ▶ ✓ Sampler TPE por padrão
- ▶ ✓ Pruning de trials não promissoras
- ▶ ✓ Dashboard visual interativo
- ▶ ✓ Suporte a otimização paralela
- ▶ ✓ API simples e flexível

➔ Consulte o Material Didático e Colab para experimentos

AutoML: Aprendizado de Máquina Automatizado

TPOT

Programação Genética para otimizar automaticamente pipelines de ML
Evolui combinações de pré-processamento, transformações e modelos

Auto-sklearn

Bayesian Optimization + *meta-learning* + *ensemble*
Aprende de resultados em datasets anteriores (warm start)

AutoGluon

Stacking multicamadas automático
Combina múltiplos modelos em estrutura de ensemble hierárquico

Consulte

Material Didático Módulo 5 e Colab para experimentos práticos com AutoML e Optuna.
Compare Grid Search e Bayesian Optimization em datasets reais.

Transformers para Dados Tabulares

FT-Transformer (Gorishniy et al., 2021)

Feature Tokenizer + Transformer

Converte features numéricas/categóricas em tokens e aplica atenção

TabPFN (Hollmann et al., 2022)

Prior-Fitted Network

Treinado em datasets sintéticos — estado da arte em datasets pequenos
Inferência em milissegundos sem fine-tuning

O Grande Debate

- ▶ ➔ Transformers atingem resultados **state-of-the-art** em benchmarks tabulares
- ▶ ➔ Porém **GBDTs** (XGBoost, LightGBM, CatBoost) ainda dominam em **eficiência e robustez** no mundo real
- ▶ ➔ Recomendação: comece com GBDT, experimente Transformers se tiver dados suficientes

Self-Supervised Learning e Redes Generativas

SCARF (Bahri et al., 2022)

Contrastive Learning para dados tabulares
Aprende representações ricas sem rótulos via corrupção de features
Pré-treino + fine-tuning com poucos exemplos rotulados

Dados Sintéticos Tabulares

- ▶ **CTGAN**: GAN condicional para tabelas
- ▶ **TVAE**: Variational Autoencoder tabular

Geram dados sintéticos realistas preservando distribuições marginais e correlações

Aplicações Práticas

- ▶ **Balanceamento**: gerar exemplos da classe minoritária
- ▶ **Privacidade**: substituir dados sensíveis por sintéticos
- ▶ **Data Augmentation**: ampliar datasets pequenos
- ▶ **Benchmarking**: criar cenários de teste controlados

Reflexão

Dados sintéticos de qualidade podem valer mais do que mais dados reais!

Aprofunde-se no Material Didático

Consulte sempre o Material Didático

Os slides são um guia didático. Para teoria completa, fórmulas e demonstrações, acesse o **Módulo 5** — **Material Didático** e os **notebooks Colab**.

- ▶ **Módulo 5:** Redes Neurais, Deep Learning, Computação Evolutiva, Fuzzy, AutoML
- ▶ **Colab:** Implementações em TensorFlow/Keras, Algoritmo Genético, PSO, Optuna
- ▶  <https://github.com/eltonsarmanho>

Referências Bibliográficas

-  GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016.
-  GÉRON, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly, 2019.
-  RUMELHART et al. Learning representations by back-propagating errors. *Nature*, 323, 1986.
-  HOLLAND, J. H. *Adaptation in Natural and Artificial Systems*. U. Michigan Press, 1975.
-  ZADEH, L. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.
-  AKIBA et al. Optuna: A next-generation hyperparameter optimization framework. *KDD*, 2019.